MeTTaDinC

Executive summary

A MeTTa sublanguage for AI systems that model and reason about their own workings and each other's
5 November 2026. Developed from the d-calculus and Hyperseed project

---

What MeTTaDinC is

MeTTaDinC is a proposed sublanguage of MeTTa, the programming language of the Hyperon AGI framework. It lets an AI system describe a part of itself, or of a peer, as a small mathematical machine, made of states, the operations that change them and the questions other components ask about them, and then put precise questions to that description. Is this compressed memory still adequate for the decisions that depend on it, now and after future updates? Does a revised goal still call for the same actions? Is a component's error rate, judged from its logs, below what was accepted? Each answer comes back as an atom that the system can store and reason over, saying what kind of evidence supports it and carrying either a replayable counterexample or, where possible, a proof that a small independent checker can verify.

Why it matters for seed AGI

A seed AGI grows by modifying itself. In Hyperon and Omega this happens continually and in small ways: memories are compressed, schemas migrated, goals revised, skills shared between agents, and agents forked and later merged. Each of these acts decides which differences the system keeps track of and which it treats as irrelevant, and a wrong decision shows up later, when something downstream needs a difference that was thrown away. MeTTaDinC gives MeTTa-based systems a simple, tractable way to make such decisions explicit and to check them, so that their self-reasoning and self-modification rest on checked statements. It is designed first for the agents themselves, Omega agents and hives, and also for the engineers who build them.

A concrete case: the first live review of the goal engine in WMTM, the memory layer of an Omega agent, rated only 18 percent of its suggestions useful or plausible, partly because settled questions kept being proposed again. Modeled in MeTTaDinC, the cause became a short counterexample. Deciding that an item is closed from what a recall window currently shows cannot be kept correct as time passes, whatever the width of the window. Recording the closing as an event passes every check, and the same analysis gives the smallest memory the engine needs.

The mathematics, and the place of Hyperseed

The language rests on the distinction calculus, mathematics developed to model minds organized as hives of self-modifying agents. Its basic objects are observers, which are judgments of which states look alike; dynamics, which test whether those judgments survive what the system does; measures of what an observer keeps and loses; and hives of observers linked by correspondences, whose coherence can be checked around cycles. MeTTaDinC makes a finite, checkable part of this mathematics usable inside running systems.

Hyperseed plays the complementary role. It is an ontology for seed AGI, a vocabulary of agents, situations, contexts, evidence, distinctions, habits and commitments, that connects an AI system to the variety of real-world situations it has to handle. MeTTaDinC connects the system's actual structures, dynamics and interactions to mathematics that can guide nontrivial reasoning about them. The two meet in shared concepts such as distinctness and interchangeability, and the specification defines how MeTTaDinC results enter Hyperseed's proof plans as evidence that keeps its status and scope. Writing Hyperseed out explicitly in MeTTaDinC is a near-term goal.

How it works

A program declares a module of machines, queries and candidate encodings, then sends requests to a checker:

```
(Representation WindowView (Of GoalEngine) (Encode window-view))
!(dc.run OpenItems (CheckUpdates WindowView))
```

Every result carries one of seven statuses. Checked means established by exact computation or a replayed proof, and Refuted means a replayed counterexample. Conditional means the claim holds if named assumptions do, such as a solver being right when its proof was not checked, and Estimated means a statistical claim from logs with a stated error guarantee. The other three mark what could not be decided, what is not yet implemented, and requests that apply a result to the wrong object. Components declare which statuses they accept, so a protected goal can insist on Checked results while a routine component accepts a trusted solver's word.

What the work so far shows

Problem	What MeTTaDinC established
Omega's goal engine	The recall-window design is refuted with replayable witnesses and event-sourced closing passes. The engine needs only nine classes of item, and serving the agent's commitment record as well takes ten.
Changing a goal	A goal replaced by its negation passes every informational check yet asks for the opposite decision on all 40 records. A separate check of meaning catches it, and proposed goal changes sort into six kinds that a promotion gate can act on.
Live memory at any scale	Summary statistics checked by solvers for every state of a store of any size, and anytime-valid estimates from logs whose one-sided bounds stay valid when error rates vary.

Problem	What MeTTaDinC established
Eight simulated weeks of the goal engine	A parser regression caught by a changepoint monitor 47 hours after it began; a promotion decision made on four claims established at level 0.01, with its conditions recorded; the next release checked before it ships.
Skill sharing in a robot hive	Transfers that are exact on every edge still return a mirrored skill around a cycle. The check names the twist and decides which symmetry declarations make the hive coherent.
Audits too large to enumerate	Two authoring mistakes found in a 40-bit access policy, and the corrected policy proved safe with 80 proofs that a verified checker replays.

MeTTa as a home for sublanguages

One of the ideas behind MeTTa from the start was that it should make domain-specific languages easy to build, and MeTTaDinC and MeTTaProN, a probabilistic and neural sublanguage, show what such languages can look like. MeTTa programs and data share one representation, atoms in an atomspace, so a sublanguage needs only a vocabulary of forms, an evaluator, and results returned as atoms. An agent can then write a declaration of its own memory, check it, store the refutation and reason about it with PLN, all in one medium. The sublanguages connect through typed interfaces rather than parallel syntax. MeTTaDinC imports MeTTaProN models as transition kernels, with exactness travelling with each kernel, and hands its results to omega-PLN and to Hyperseed's proof plans.

Status and next steps

No MeTTaDinC runtime exists yet. Every result in the documents was computed from the module text by a reference evaluator written in Python and checked again against independent Python models; the validation oracles run 157, 305, 195, 94 and 99 checks, all passing. An implementation guide hands the work to coding agents in eight milestones, hosted in Cetta, the C runtime for MeTTa, starting with strict boundaries on results and a deliberately narrow first release. The next steps are that first release, a MeTTaDinC layer beside Omega's memory and goal machinery whose results feed the GOLEM-Iter promotion gate, and an explicit rendering of Hyperseed in MeTTaDinC.

The documents

- The MeTTaDinC language specification, with its validation package (86 pages)
- Implementing MeTTaDinC in Cetta: the coding-agent work plan (30 pages)
- MeTTaDinC in Practice: four worked examples (79 pages)
- Omega Memory and Goals in MeTTaDinC, Part I: implementation guide (66 pages)
- Part II: live memory through summary statistics (53 pages)
- Worked example: WMTM's open-item store, from journal to gate (45 pages)
- MeTTaDinC: an expository introduction (18 pages), of which this is the summary